

Week 3 Assignment: PPO on CartPole and a Toy Trading Environment

Overview

In this assignment you will do three things:

1. **Probe REINFORCE's failure modes** by deliberately choosing bad hyperparameters and observing what breaks.
2. **Train a PPO agent** on CartPole using Stable-Baselines3, compare its reward curve to your Week 2 REINFORCE result, and explain the difference.
3. **Build and test a minimal custom trading environment** using Gymnasium's interface, then run a random agent on it.

Setup

Install the one new dependency before starting:

```
pip install stable-baselines3
```

Your Week 2 packages (PyTorch, Gymnasium, NumPy, Matplotlib) are still required. Confirm everything still imports without errors before starting Task 1.

Project Structure

By the end of Week 3, your folder should look like:

```
project_root/
├── week2_reinforce.py
├── week2_notes.md
├── week3_probe.py
├── week3_ppo.py
├── week3_compare.py
├── week3_trading_env.py
├── week3_notes.md
├── plots/
│   ├── cartpole_rewards.png           (from Week 2)
│   └── reinforce_vs_ppo.png          (Task 3)
```

Task 1: Probe REINFORCE's Failure Modes (Core)

Create `week3_probe.py`. Starting from your Week 2 REINFORCE training function, run three additional experiments by changing one hyperparameter at a time:

Run	learning_rate	gamma	Expected effect
Baseline (Week 2)	1e-2	0.99	Normal learning
Run A	0.1	0.99	Too large — diverge?
Run B	1e-5	0.99	Too small — stuck?
Run C	1e-2	0.50	Short-sighted?

Run each for 300 episodes. Log the episode returns. You do not need to plot them (yet) — just print the final 10-episode average for each run.

What You Must Do

1. Copy (or import) `train_reinforce` from your Week 2 file.
2. Run it four times with the hyperparameters above.
3. Print, for each run: the hyperparameters used and the mean return over the final 10 episodes.

Task 2: Train PPO on CartPole (Core)

Create `week3_ppo.py` using this skeleton:

```
# week3_ppo.py

import gymnasium as gym
from stable_baselines3 import PPO
from stable_baselines3.common.evaluation import evaluate_policy
from stable_baselines3.common.callbacks import BaseCallback
import numpy as np

class EpisodeRewardLogger(BaseCallback):
    """Records the total reward of each completed episode."""

    def __init__(self):
        super().__init__()
        self.episode_rewards = []
        self._current_reward = 0.0

    def _on_step(self) -> bool:
        self._current_reward += self.locals["rewards"][0]
        if self.locals["dones"][0]:
            self.episode_rewards.append(self._current_reward)
            self._current_reward = 0.0
        return True # continue training

def train_ppo(env_name="CartPole-v1", total_timesteps=50_000):
    env = gym.make(env_name)

    callback = EpisodeRewardLogger()

    model = PPO(
        "MlpPolicy",
```

```

        env,
        verbose=0,
        learning_rate=3e-4,
        n_steps=2048,
        batch_size=64,
        gamma=0.99,
    )
    model.learn(total_timesteps=total_timesteps, callback=callback)

    mean_reward, std_reward = evaluate_policy(model, env, n_eval_episodes
=10)
    print(f"PPO evaluation: {mean_reward:.1f} +/- {std_reward:.1f}")

    env.close()
    return callback.episode_rewards

if __name__ == "__main__":
    ppo_returns = train_ppo()

```

What You Must Do

1. Read through the skeleton and add a comment on each of these lines explaining what it does in your own words:
 - `n_steps=2048`
 - `batch_size=64`
 - The `EpisodeRewardLogger` class
2. Run the script for at least 50 000 timesteps and confirm the final evaluation mean reward is above 400.
3. Save `ppo_returns` to a file (e.g., using `numpy.save`) so Task 3 can load it.

Task 3: Compare REINFORCE and PPO (Core)

Create `week3_compare.py`. Load the episode returns from your Week 2 REINFORCE run and the PPO run from Task 2, then produce a single comparison plot.

```

# week3_compare.py

import numpy as np
import matplotlib.pyplot as plt
import os

def moving_average(x, window=10):
    if len(x) < window:
        return np.array(x)
    return np.convolve(x, np.ones(window) / window, mode="valid")

```

```

def plot_comparison(reinforce_returns, ppo_returns,
                   window=10,
                   filename="plots/reinforce_vs_ppo.png"):
    os.makedirs("plots", exist_ok=True)

    fig, ax = plt.subplots(figsize=(9, 4))

    # REINFORCE
    r = np.array(reinforce_returns)
    ax.plot(r, alpha=0.2, color="steelblue")
    ax.plot(moving_average(r, window), color="steelblue",
            label=f"REINFORCE ({window}-ep avg)")

    # PPO
    p = np.array(ppo_returns)
    ax.plot(p, alpha=0.2, color="darkorange")
    ax.plot(moving_average(p, window), color="darkorange",
            label=f"PPO ({window}-ep avg)")

    ax.set_xlabel("Episode")
    ax.set_ylabel("Return")
    ax.set_title("REINFORCE vs PPO on CartPole-v1")
    ax.legend()
    fig.tight_layout()
    fig.savefig(filename)
    plt.close(fig)
    print(f"Saved to {filename}")

if __name__ == "__main__":
    # Load your saved returns --- adjust filenames as needed
    reinforce_returns = np.load("reinforce_returns.npy").tolist()
    ppo_returns = np.load("ppo_returns.npy").tolist()
    plot_comparison(reinforce_returns, ppo_returns)

```

What You Must Do

1. Ensure both training runs save their returns with `numpy.save`.
2. Run `week3_compare.py` and verify the plot is saved at `plots/reinforce_vs_ppo.png`.

Task 4: Build a Toy Trading Environment (Core)

Create `week3_trading_env.py` using this skeleton:

```

# week3_trading_env.py

import gymnasium as gym
from gymnasium import spaces
import numpy as np

```

```

class ToyTradingEnv(gym.Env):
    """
    Simplest possible trading environment.

    Dynamics:
    - Asset price follows a random walk (log-normal returns).
    - Episode length is fixed at 'episode_length' steps.

    Observation (2-dimensional):
    [last_price_return, current_position]
    position in {-1.0 (short), 0.0 (flat), 1.0 (long)}

    Actions (discrete):
    0 = Go Long    (buy / stay long)
    1 = Hold       (keep current position)
    2 = Go Short   (sell / stay short)

    Reward:
    current_position * price_return * 100
    (profit if position matches price direction, loss otherwise)
    """

    def __init__(self, episode_length=100, initial_price=100.0,
                  daily_vol=0.01):
        super().__init__()
        self.episode_length = episode_length
        self.initial_price = initial_price
        self.daily_vol = daily_vol

        self.observation_space = spaces.Box(
            low = np.array([-np.inf, -1.0], dtype=np.float32),
            high = np.array([ np.inf,  1.0], dtype=np.float32),
        )
        self.action_space = spaces.Discrete(3) # 0=long, 1=hold, 2=short

        self.price = None
        self.position = None
        self.step_count = None

    # ----- #
    def reset(self, seed=None, options=None):
        super().reset(seed=seed)
        self.price = self.initial_price
        self.position = 0.0 # start flat
        self.step_count = 0

        obs = np.array([0.0, self.position], dtype=np.float32)
        return obs, {}

    # ----- #
    def step(self, action):
        # Price dynamics
        price_return = float(self.np_random.normal(0.0, self.daily_vol))

```

```

self.price *= (1.0 + price_return)

# Map action new position
action_to_position = {0: 1.0, 1: self.position, 2: -1.0}
new_position = action_to_position[int(action)]

# Reward: did the old position benefit from this price move?
reward = self.position * price_return * 100.0

# Advance state
self.position = new_position
self.step_count += 1

terminated = self.step_count >= self.episode_length
truncated = False

obs = np.array([price_return, self.position], dtype=np.float32)
return obs, reward, terminated, truncated, {}

# ----- #
def render(self):
    print(f"Step {self.step_count:3d} | "
          f"Price {self.price:8.2f} | "
          f"Position {self.position:+.0f}")

# #
def run_random_agent(n_episodes=10):
    """Run a random agent and report average total reward."""
    env = ToyTradingEnv()
    total_rewards = []

    for ep in range(n_episodes):
        obs, _ = env.reset()
        total_reward = 0.0
        done = False
        while not done:
            action = env.action_space.sample()
            obs, reward, terminated, truncated, _ = env.step(action)
            total_reward += reward
            done = terminated or truncated
        total_rewards.append(total_reward)
        print(f"Episode {ep + 1:2d}: total reward = {total_reward:7.2f}")

    print(f"\nMean over {n_episodes} episodes: "
          f"{np.mean(total_rewards):.2f}")
    env.close()
    return total_rewards

if __name__ == "__main__":
    run_random_agent()

```

What You Must Do

1. Read through the skeleton. Add comments in your own words explaining:
 - Why `price_return` is multiplied by `self.position` in the reward.
 - What happens to the agent's position when `action = 1` (Hold).
 - What `self.np_random.normal` is doing and where the random seed comes from.
2. Run the script and record the mean reward over 10 episodes.
3. Confirm the environment passes a basic sanity check:

```
from gymnasium.utils.env_checker import check_env
check_env(ToyTradingEnv(), warn=True)
```

If `check_env` raises warnings, fix them before proceeding.

Task 5: Written Reflection (`week3_notes.md`)

Create `week3_notes.md` and answer each question in 2–4 sentences.

On REINFORCE failure modes (Task 1):

1. Which hyperparameter change hurt learning the most? What does this tell you about gradient-based optimisation?
2. REINFORCE updates after every single episode. Why does using one trajectory's data make the gradient estimate noisy?

On PPO (Task 2):

3. PPO uses a clipped probability ratio. In your own words, what problem does the clip prevent?
4. PPO trains on mini-batches from a large replay buffer. Why might this produce a smoother reward curve than REINFORCE?

On the comparison (Task 3):

5. Describe the two reward curves in 3–4 sentences. Which learned faster? Which was more consistent? Were there any surprises?

On the trading environment design (general):

6. List 5 features you would add to the state to make the environment more realistic. For each, explain why it would help the agent.
7. The current reward is `position * price_return * 100`. What happens if the agent always stays long? Is that a good strategy on a random walk?
8. What are three limitations of this toy environment compared to a real financial market? How might each limitation affect what the agent learns?

On the trading environment code (Task 4):

9. What was the mean reward of the random agent over 10 episodes? Was it positive, negative, or near zero? Does that match your expectation?

10. Write down **3 questions** about environment design or RL-for-trading that you want to discuss with your mentor.

Optional Extension: Train PPO on the Trading Environment

If you are ahead of pace, pass `ToyTradingEnv()` to PPO and train for 100 000 timesteps:

```
from stable_baselines3 import PPO
from week3_trading_env import ToyTradingEnv

env = ToyTradingEnv()
model = PPO("MlpPolicy", env, verbose=1,
            learning_rate=3e-4, n_steps=512, batch_size=64)
model.learn(total_timesteps=100_000)
```

Does the agent beat the random baseline from Task 4? Plot the reward curve and include a brief note in `week3_notes.md` on what you observe.

Deliverables

By the end of Week 3 you should have:

- **Code:** `week3_probe.py`, `week3_ppo.py`, `week3_compare.py`, `week3_trading_env.py`.
- **Notes:** `week3_notes.md` with answers to all 10 questions.
- **Plot:** `plots/reinforce_vs_ppo.png`.

During your mentor meeting, walk through:

- The REINFORCE vs. PPO comparison plot — what story does it tell?
- Your state/action/reward design choices for the trading environment and any limitations you identified.
- The 3 questions from Question 10 in your notes.